NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

Experiment:- 7

Aim: To implement monitoring using Prometheus / Grafana / Nagios.

LO mapped: LO5

LO: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

Theory:

Continuous monitoring is a process to detect, report, respond all the attacks which occur in its infrastructure. Once the application is deployed into the server, the role of continuous monitoring comes in to play. The entire process is all about taking care of the company's infrastructure and respond appropriately.

Why we need Nagios Tool

Here, are the important reasons to use Nagios monitoring tool:

- Detects all types of network or server issues
- Helps you to find the root cause of the problem which allows you to get the permanent solution to the problem
- Active monitoring of your entire infrastructure and business processes
- Allows you to monitor and troubleshoot server performance issues
- Helps you to plan for infrastructure upgrades before outdated systems create failures
- You can maintain the security and availability of the service
- Automatically fix problems in a panic situation
- Nagios is the most popular, open source, powerful monitoring system for any kind of infrastructure. It enables organizations to identify and resolve IT infrastructure problems before they affect critical business processes. Nagios has the capability of monitoring applications, services and entire IT infrastructure.
- NRPE is known as Nagios Remote Plugin Executor. It is designed to execute plugins on remote systems. On Windows hosts, the equivalent remote agent is NSClient++, which listens for Nagios requests and executes local checks (disk, CPU, memory, services, etc.).
- NSClient++ runs as a Windows service on the remote host and waits for requests from the Nagios server (usually over the NRPE protocol, which NSClient++ also supports). When the Nagios server needs to check a resource on that host, it sends a command; NSClient++ executes the check locally and returns the result to Nagios.

Note on running Nagios on Windows

Nagios Core is built for Linux and does not install natively on Windows. The standard practice for a Windows machine is to run the Nagios Core server inside a Docker container (using Docker Desktop with the WSL2 backend), and to monitor Windows hosts using NSClient++, which is the Windows

NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

counterpart of NRPE. All commands below are Windows PowerShell / Command Prompt commands.

1: Pre-requisite: Install Docker Desktop

Download and install Docker Desktop for Windows and enable the WSL2 based engine. Verify the installation from PowerShell:

```
PS C:\Users angad > docker --version
PS C:\Users angad > docker version
```

Also make sure the WSL2 feature is enabled:

```
PS C:\Users angad > wsl --install
PS C:\Users angad > wsl --set-default-version 2
```

2: Pull and Run the Nagios Core Docker Image

Pull a ready-made Nagios Core image and run it as a container, mapping the web port to the host.

```
PS C:\Users angad > docker pull jasonrivers/nagios:latest
PS C:\Users angad > docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest
```

Verify the container is up and running:

```
PS C:\Users angad > docker ps
```

3: Set the Apache/Nagios Admin Password

Set (or reset) the password for the nagiosadmin user inside the running container:

```
PS C:\Users angad > docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
```

Enter and confirm the password when prompted (e.g. 123456).

4. Access Nagios Web Interface

Access the Nagios web interface from a browser on the Windows host:

```
http://localhost:8080/nagios/
```

Prompting for Authentication Password - username:

nagiosadmin

Password: 123456 (which you entered while configuration)

NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

5: Install NSClient++ on the Windows Remote Host

On the Windows machine you want to monitor, download and silently install NSClient++ (the Windows equivalent of the NRPE agent):

```
PS C:\Users angad > Invoke-WebRequest -Uri
"https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi" -OutFile
"NSCP.msi"
PS C:\Users angad > Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL
MODULES=CheckExternalScripts,NRPEServer /qn' -Wait
```

6: Configure NSClient++

Edit the NSClient++ configuration file and add the Nagios server's IP address to the allowed hosts list:

```
PS C:\Program Files\NSClient++> notepad nsclient.ini allowed
hosts=127.0.0.1,192.168.1.100
```

Where 192.168.1.100 is your Nagios server (Docker host) IP address.

After saving the file, restart the NSClient++ service so the change takes effect:

```
PS C:\Users angad > Restart-Service nscp
```

7: Verify Connectivity from Nagios

From the Nagios server (inside the Docker container), run the NRPE check against the Windows remote host to confirm connectivity. Here 192.168.1.50 is the remote Windows host's IP address.

```
PS C:\Users angad > docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H 192.168.1.50
NRPE v2.15
```

8: Add the Windows Host in Nagios

Create a configuration file for the new host inside the Nagios container. We also define a service that continuously checks whether the host is reachable (PING check).

NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

```
PS C:\Users angad > docker exec -it nagios4 sh -c "vi
/opt/nagios/etc/servers/MyWindowsHost001.cfg"
define host {     use             windows-server
host_name       Windows_Host_001     alias
Windows Host 001     address          192.168.1.50
register        1
} define service{     host_name             Windows_Host_001
service_description    PING     check_command
check_ping!100.0,20%!500.0,60%
```

```
   max_check_attempts     2
check_interval        2
retry_interval        2
check_period          24x7
check_freshness       1
contact_groups        admins
notification_interval  2
notification_period    24x7
notifications_enabled  1
   register             1 }
```

Verify the configuration file for errors, then restart the Nagios container so the changes take effect:

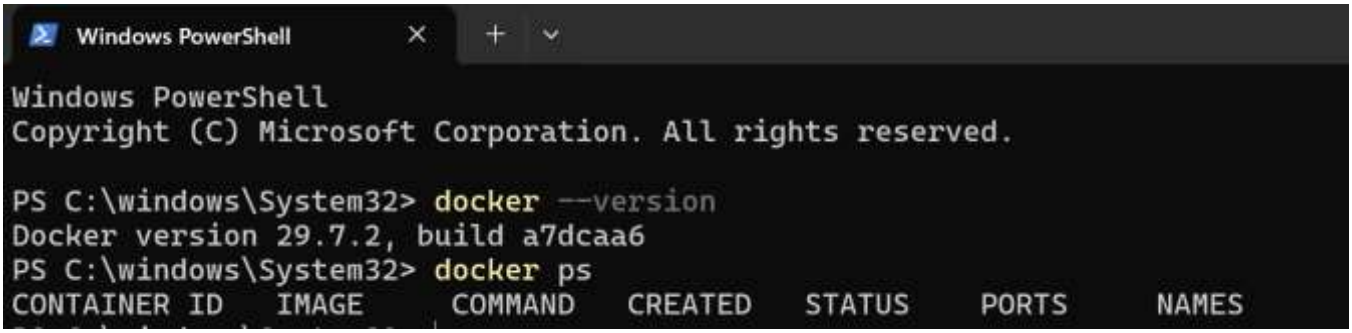
```
PS C:\Users angad > docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg PS
C:\Users angad > docker restart nagios4
```

9: Check Host in Nagios Web Interface

Open the Nagios web interface (http://localhost:8080/nagios/) and check that the new Windows host has been added and is being monitored under Current Status -> Hosts.

NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

Output:



NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

```
PS C:\windows\System32> docker pull jasonrivers/nagios:latest
latest: Pulling from jasonrivers/nagios
a80db686d641: Pull complete
1b9771f55a12: Pull complete
254354b70ac0: Pull complete
a6b90b5bbb15: Pull complete
fdbaa15b8986: Pull complete
a03720c2c16b: Pull complete
f2f9a1b3ce48: Pull complete
50ea270d34e9: Pull complete
e4dc1d4e010c: Pull complete
cb259a83ac3d: Pull complete
9288c4992679: Pull complete
f47c3af6bf44: Pull complete
37f75c8146e2: Pull complete
7a7447982198: Pull complete
e68fcda12c0c: Pull complete
c42073aaf047: Pull complete
748b632e75e7: Pull complete
80b6ab7aef5d: Pull complete
1413dee23aae: Pull complete
c5701b5e1540: Pull complete
0d59a81a5608: Pull complete
59215b8e47cf: Pull complete
54b9e993e315: Pull complete
a7bd6dce6a77: Pull complete
6fe99ef1bed3: Pull complete
d003d4303177: Pull complete
7edde1931fd2: Pull complete
2908452ff193: Pull complete
dc3a88012a3e: Pull complete
Digest: sha256:fee9cd6a2933e790dde613d70c3a603f96978d65fa6bbc3d617fce807e00f2e2
Status: Downloaded newer image for jasonrivers/nagios:latest
docker.io/jasonrivers/nagios:latest
```

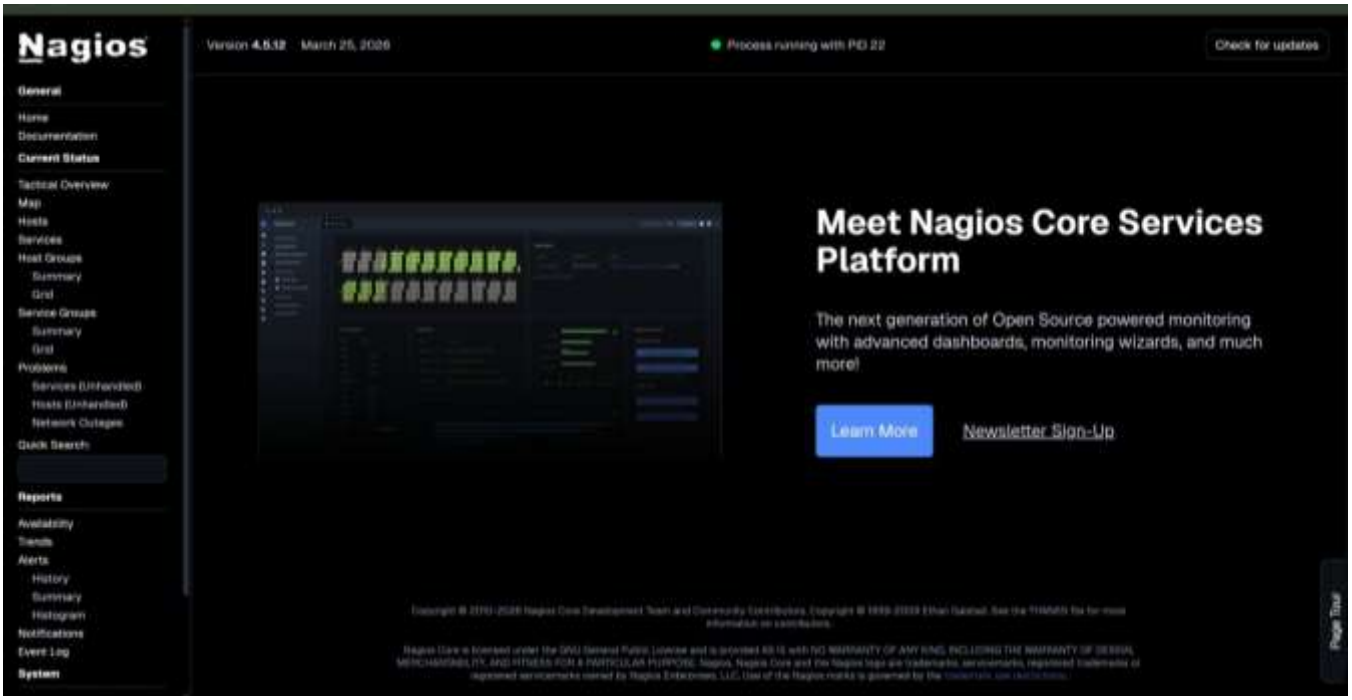
```
PS C:\windows\System32> docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest
c3271a2781e419515ba45281ea881162a5385aeb88319bd561c8e8309edbfa1e
PS C:\windows\System32> docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED         STATUS         PORTS                                     NAMES
c3271a2781e4   jasonrivers/nagios:latest   "/usr/local/bin/star…"   21 seconds ago   Up 18 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   nagios4
```

```
PS C:\windows\System32> docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
New password:
Re-type new password:
Adding password for user nagiosadmin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
```



NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D




```
PS C:\windows\System32> ipconfig

Windows IP Configuration


Unknown adapter McAfee VPN:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi 2:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi 3:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

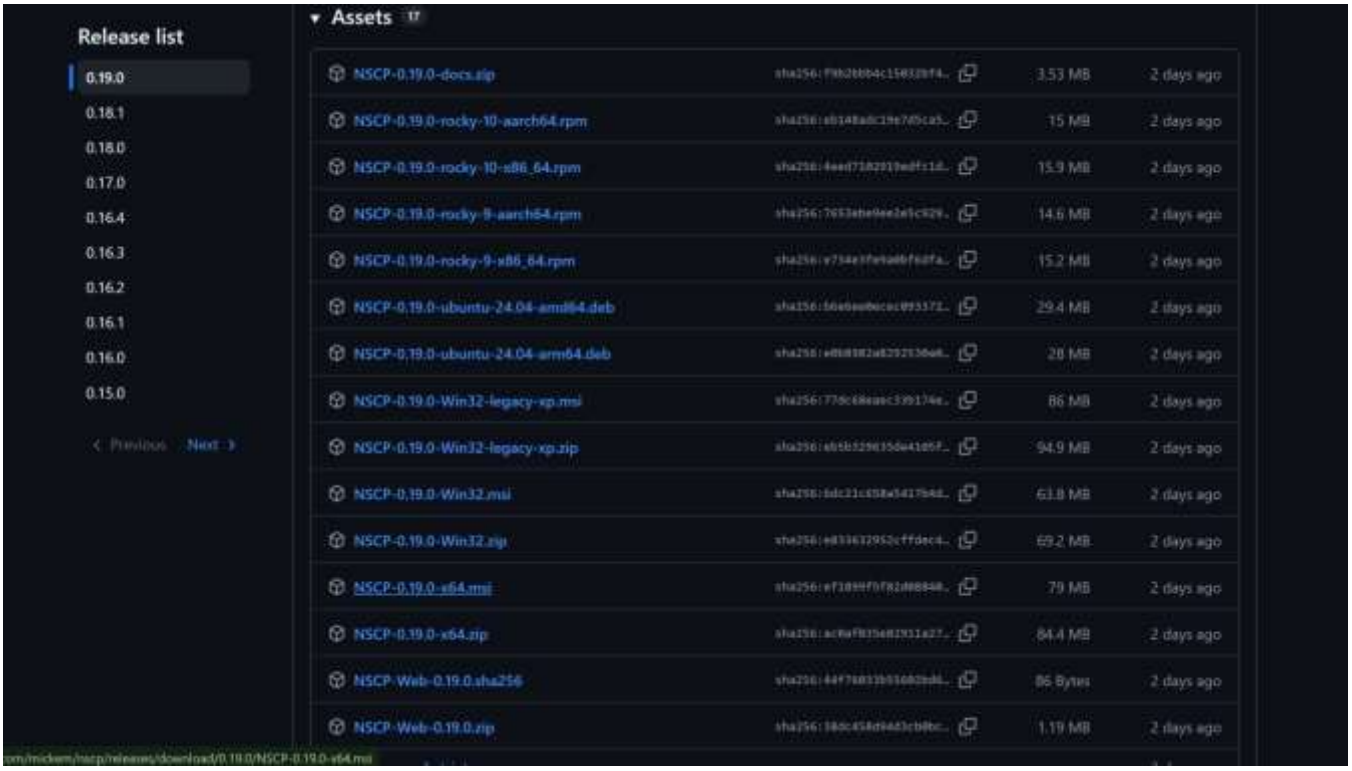
Wireless LAN adapter Wi-Fi:

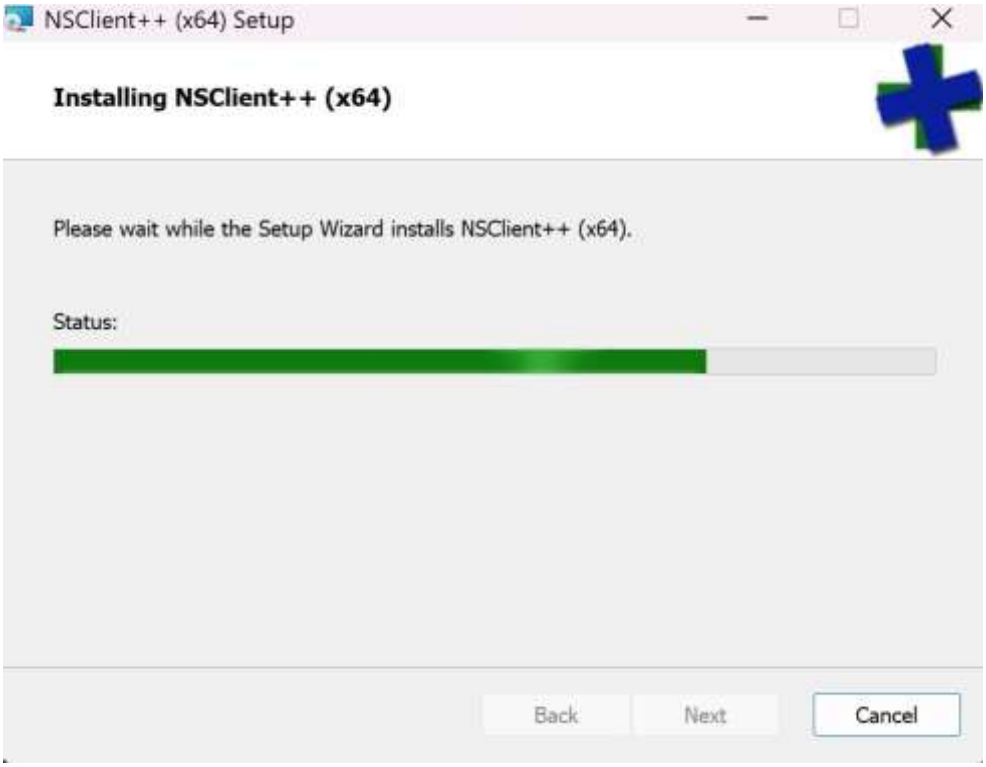
   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2405:201:15:c129:f744:701f:fd9a:9016
   Temporary IPv6 Address. . . . . . : 2405:201:15:c129:d524:2e7c:765d:edaf
   Link-local IPv6 Address . . . . . : fe80::ab17:de5e:5397:65d4%16
   IPv4 Address. . . . . . . . . . . : 192.168.29.181
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::f6ca:e7ff:fef2:e3c%16
                                       192.168.29.1

Ethernet adapter vEthernet (WSL (Hyper-V firewall)):

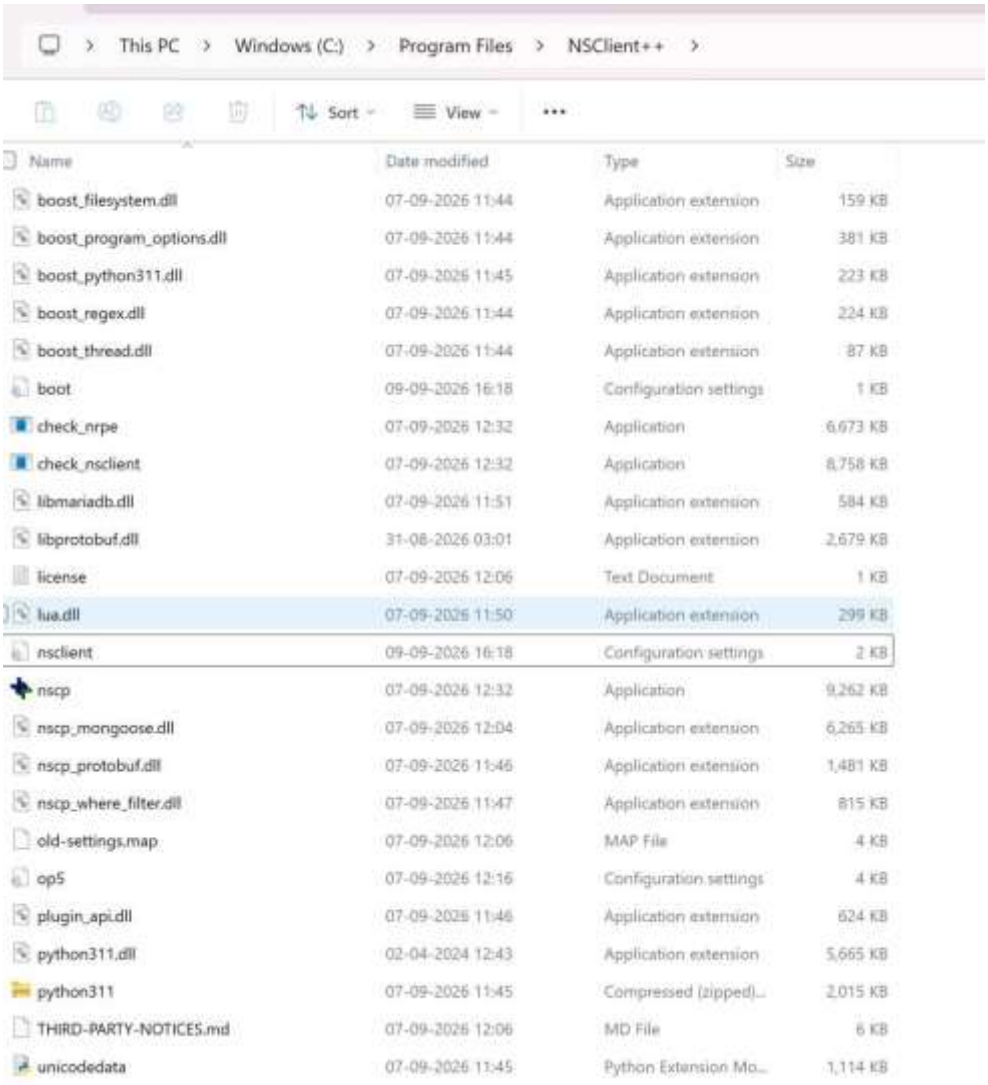
   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::f2f:37e2:f0c6:fc97%46
   IPv4 Address. . . . . . . . . . . : 172.17.16.1
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . :
```

NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D





NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D




```
CheckHelpers = enabled

; Load on startup - If the module should be loaded on startup
CheckExternalScripts = enabled

; Load on startup - If the module should be loaded on startup
CheckNSCP = enabled


; in flight - TODO
[/settings/NRPE/server]

verify mode = peer-cert

tls version = tlsv1.2+

insecure = false


; in flight - TODO
[/settings/default]

allowed hosts = 127.0.0.1,192.168.29.181,172.17.16.1,172.17.0.0/16

password = vOiMR4smBNLkJBxs
```

NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

```
PS C:\windows\system32> notepad "C:\Program Files\NSClient++\nsclient.ini"
PS C:\windows\system32> Restart-Service nscp
PS C:\windows\system32>
```

```
PS C:\windows\system32> New-NetFirewallRule -DisplayName "Nagios NRPE" -Direction Inbound -LocalPort 5666 -Protocol TCP -Action Allow


Name                            : {494a7a4a-65cd-4bde-8907-5b4ddb608de0}
DisplayName                     : Nagios NRPE
Description                     :
DisplayGroup                    :
Group                           :
Enabled                         : True
Profile                         : Any
Platform                        : {}
Direction                       : Inbound
Action                          : Allow
EdgeTraversalPolicy             : Block
LooseSourceMapping              : False
LocalOnlyMapping                : False
Owner                           :
PrimaryStatus                   : OK
Status                          : The rule was parsed successfully from the store. (65536)
EnforcementStatus               : NotApplicable
PolicyStoreSource               : PersistentStore
PolicyStoreSourceType           : Local
RemoteDynamicKeywordAddresses   : {}
PolicyAppId                     :
PackageFamilyName               :



PS C:\windows\system32> docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H host.docker.internal -n
l (0.19.0 2026-09-07) seem to be doing fine...

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
```

```
PS C:\windows\system32> docker exec -it nagios4 sh -c "mkdir -p /opt/nagios/etc/servers && echo 'define host { use windows-server; host_name Windows_Host_001; alias Windows Host 001; address
host.docker.internal; register 1; } define service { host_name Windows_Host_001; service_description PING; check_command check_ping!100.0,20%!500.0,60%; max_check_attempts 3; check_interva
l 2; retry_interval 2; check_period 24x7; check_freshness 1; contact_groups admins; notification_interval 2; notification_period 24x7; notifications_enabled 1; register 1; }' > /opt/nagios/
etc/servers/MyWindowsHost001.cfg"
PS C:\windows\system32> docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
       Checked 7 services.
       Checked 1 hosts.
       Checked 1 host groups.
       Checked 0 service groups.
       Checked 1 contacts.
       Checked 1 contact groups.
       Checked 23 commands.
       Checked 5 time periods.
       Checked 0 host escalations.
       Checked 0 service escalations.
Checking for circular paths...
       Checked 1 hosts
       Checked 0 service dependencies
       Checked 0 host dependencies
       Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
```

```
PS C:\windows\system32> docker restart nagios4
nagios4
```

NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

```
>> '@ | Set-Content -Path ".\MyWindowsHost001.cfg"
PS C:\Windows\system32> docker cp .\MyWindowsHost001.cfg nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
Successfully copied 9948 (transferred 2.56kB) to nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
PS C:\Windows\system32> docker restart nagios4
nagios4
PS C:\Windows\system32> @'
>> define command {
>>     command_name    check_nrpe_alive
>>     command_line    $USER1$/check_nrpe -H $HOSTADDRESS$ -n
>> }
>>
>> define host {
>>     use                     generic-host
>>     host_name               Windows_Host_001
>>     alias                   Windows Host 001
>>     address                 host.docker.internal
>>     check_command           check_nrpe_alive
>>     max_check_attempts      1
>>     check_interval          1
>>     retry_interval          1
>>     check_period            24x7
>>     notification_interval   2
>>     notification_period     24x7
>>     notifications_enabled   1
>>     register                1
>> }
>>
>> define service {
>>     use                     generic-service
>>     host_name               Windows_Host_001
>>     service_description     NSClient_Health
>>     check_command           check_nrpe_alive
>>     max_check_attempts      1
>>     check_interval          1
>>     retry_interval          1
>>     check_period            24x7
>>     check_freshness         1
>>     contact_groups          admins
>>     notification_interval   2
>>     notification_period     24x7
>>     notifications_enabled   1
>>     register                1
>> }
>> '@ | Set-Content -Path ".\MyWindowsHost001.cfg"
PS C:\Windows\system32> docker cp .\MyWindowsHost001.cfg nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
Successfully copied 1.11kB (transferred 3.07kB) to nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
PS C:\Windows\system32> docker restart nagios4
nagios4
```



NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D



Conclusion:

In this experiment, continuous infrastructure monitoring was implemented using Nagios Core and NSClient++. Nagios Core was successfully containerized using Docker on Windows via WSL2, bypassing native operating system constraints. Communication across the container-to-host boundary was established over the NRPE protocol on port 5666. Using custom host and service definitions, remote Windows host health metrics were gathered and verified in real time on the Nagios web dashboard, demonstrating how centralized monitoring provides proactive fault detection and system visibility.

LO 5 mapped successfully.